Linear Algebra & DL

Kim youngeun
Jeonju university, Korea

DL basic architecture

$$\mathbf{y} = W\mathbf{x} + \mathbf{b}$$

single neural network layer

선형변환

$$\mathbf{a} = \sigma(\mathbf{y})$$

비선형성

ReLU, sigmoid

x: 입력 벡터

W: 가중치 행렬

b: 편향

y: 출력

Neural Network

$$\mathbf{a}^{(2)} = \sigma(W^{(2)}\sigma(W^{(1)}\mathbf{x} + b^{(1)}) + b^{(2)})$$

행렬 곱의 중첩

→ 결국 선형대수 연산의 반복 구조

Loss Function

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

모델이 얼마나 틀렸는지 측정

행렬 미분 (Gradient)

딥러닝의 목적

L(W)를 최소화

이를 위해 미분

scalar to vector derivative

$$\frac{\partial L}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial L}{\partial x_1} \\ \frac{\partial L}{\partial x_2} \end{bmatrix}$$

gradient vector

vector to matrix derivative

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial W}$$

$\mathbf{y} = W\mathbf{x}$ 이면 결과는 $\frac{\partial L}{\partial W} = \delta \mathbf{x}^T$

$$\delta = \frac{\partial L}{\partial \mathbf{y}}$$

outer product 형태

Backpropagation

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(z^{(l)})$$

δ: 오차 신호

⊙: element-wise 곱

뒤에서 앞으로 gradient 전파

Gradient Descent

$$W := W - \eta \frac{\partial L}{\partial W}$$

η: 학습률

반복하면서 최적값으로 수렴

Overall flow

1. Forward $x \to Wx + b \to \sigma \to \hat{y}$
2. Loss 계산 $L(y, \hat{y})$
3. Backward *체인룰 사용, 행렬 미분 수행*
4. Update *gradient descent*

Point

행렬 W : 공간을 변형하는 함수

미분 : 그 변형을 어떻게 수정할지 방향 알려줌

역전파 : 오차를 거꾸로 전달하는 과정

Point

CNN → convolution = 행렬 연산

Transformer → attention = 내적 + softmax

모든 학습은 gradient 기반 최적화

Linear Algebra & DL

Kim youngeun